技术文档

目录

1.数据采集技术详解.....1
1.1 爬虫逻辑介绍.....1
1.2 运行环境配置.....1
1.3 数据采集逻辑详解.....2
1.3.1 网络请求与会话管理.....2
1.3.2 智能参数组合轮询机制.....2
1.3.3 内容过滤与去重策略.....3
1.3.4 文件下载与存储管理.....3
1.3.5 反爬虫与请求优化.....3
1.3.6 异常处理与日志监控.....4
1.3.7 流程控制与性能优化.....4
1.4 总结.....4
2.结构化提取技术详解.....5
2.1 文本解析与预处理.....5
2.1.1 路径与 OCR 设置.....5
2.1.2 正文文本抽取.....5
2.1.3 文本标准化与正则友好化.....6
2.2 表格优先的要素抽取.....6
2.2.1 标签映射与键值对抽取.....6
2.2.2 费用表解析与标准化.....7
2.3 正文兜底（文本检索与规则融合）.....8
2.3.1 名称与代码.....8
2.3.2 发行时间（日期归一）.....8
2.3.3 基金类型与运作方式.....9
2.3.4 募集规模与三费率（正文兜底）.....9
2.4 机构名称识别与角色消歧（管理人/托管人）.....9
2.4.1 极端断字与跨行.....9
2.4.2 银行词典与附近窗口.....10
2.4.3 清洗与校验、角色纠偏.....10
2.5 字段后处理与一致性校验.....10
2.6 结果导出与文件说明.....11
2.7 使用建议与排错清单（可选增强）.....12

1.数据采集技术详解

1.1 爬虫逻辑介绍

本项目的数据采集任务聚焦于从巨潮资讯网（www.cninfo.com.cn）爬取基金发行公告的 PDF 文档。基金发行公告类型包括招募说明书、基金合同、发售公告、托管协议等，这些公告以 PDF 格式发布，包含基金名称、发行时间、基金管理人、托管人、费率等关键结构化信息。

爬虫逻辑分为两类数据获取方式：

静态数据请求：通过模拟浏览器 POST 请求获取公告列表 JSON 数据，解析出每条公告的标题、时间、PDF 链接等信息。

动态文件下载：根据解析得到的 PDF 链接，发起 HTTP 请求下载 PDF 文件，并进行本地存储与去重处理。

本项目采用 Python 编写爬虫程序，通过 requests 库发送网络请求，配合重试机制、随机 User-Agent、IP 代理池模拟等策略，提升爬虫的稳定性和反反爬能力。

1.2 运行环境配置

在本项目的数据采集过程中，所使用的 Python 库各司其职，共同保障了爬虫的稳定性、效率与数据质量。核心网络请求由 requests 库完成，它负责向巨潮资讯网的 API 接口发送 POST 请求以获取公告列表，并发起 GET 请求流式下载 PDF 文件；其底层的 urllib3 库通过配置重试策略和连接池，为网络通信提供了高可用的保障。返回的 JSON 数据由内置库 json 解析为 Python 字典，从而提取出公告标题、链接、时间等关键字段。为了适配本地文件系统，os 库承担了创建存储目录、检查文件是否存在（实现去重）以及合规拼接文件路径的任务；而 re 库则负责清洗文件名，通过正则表达式移除标题中的 HTML 标签和非法字符，确保文件能安全存储。反爬虫策略通过 random 和 time 库实现，前者从预定义的 User-Agent 池中随机选择浏览器标识以模拟不同设备，后者则在请求间插入随机延时，有效降低访问频率，规避 IP 封禁风险。整个过程的运行状态由 logging 库详细记录，提供从信息提示到错误追踪的全方位日志输出；此外，datetime 库用于转换时间戳为可读日期并用于文件命名，urllib.parse 则确保

了相对 URL 至绝对地址的正确拼接。这些库的协同工作，构建了一个高效、稳定且健壮的数据采集 pipeline。以下是环境详细配置：

表 1 环境配置表

配置类别	名称	版本	用途
基本环境	Python	3.12.0	运行环境
第三方库	requests	2.32.4	发送 HTTP 请求
第三方库	urllib3	2.5.0	连接池与重试机制
内置库	json	2.0.9	解析 API 返回数据
内置库	re	2.2.1	正则表达式处理文本
内置库	os	/	文件与目录操作
内置库	time	/	时间控制与延时
内置库	random	/	随机数生成
内置库	logging	/	日志记录
内置库	datetime	/	时间处理
内置库	urllib.parse	/	URL 编码与拼接

1.3 数据采集逻辑详解

1.3.1 网络请求与会话管理

为实现高效稳定的数据采集，本项目采用了基于 requests 库的会话管理机制。通过创建持久化会话对象，显著降低了重复建立 TCP 连接的开销。特别设计的重试策略能够自动处理服务器端返回的 429、500、502 等异常状态码，在遇到网络波动或服务器繁忙时，系统会自动进行最多 5 次重试，且重试间隔采用指数退避算法，有效避免了因短暂故障导致的数据采集中断。连接池大小设置为 100 个连接，充分满足了高并发请求的需求，确保了数据采集过程的高效性。

1.3.2 智能参数组合轮询机制

针对巨潮资讯网复杂的接口参数体系，本项目创新性地设计了多参数组合轮询策略。通过定义 10 种不同的 column 和 category 参数组合，全面覆盖了网站中基金公告的各种分类方式。系统采用循环遍历方式依次尝试每种参数组合，当某

种组合请求失败时自动标记并跳过，成功时则继续使用该组合获取后续页码数据。这种智能轮询机制不仅大幅提高了数据采集的覆盖率，还能自适应网站的接口变化，增强了系统的健壮性和兼容性。

1.3.3 内容过滤与去重策略

本系统实现了双层内容过滤机制，通过包含关键词列表（如"招募说明书"、"基金合同"等）和排除关键词列表（如"经理变更"、"分红"等）的协同工作，精准识别目标基金公告。同时，采用基于标题内容和 URL 地址的双重去重机制，使用集合数据结构记录已处理公告的特征值，有效避免了重复下载。在文件存储层面，系统会预先检查本地文件是否存在，确保不会重复保存相同内容，既节约了存储空间，又提高了数据处理效率。

1.3.4 文件下载与存储管理

PDF 文档的下载采用流式处理方式，通过设置 chunk_size 为 8192 字节，实现了大文件的分块下载和写入，有效避免了内存溢出的风险。在文件保存前，系统会对文件名进行规范化处理，使用正则表达式移除标题中的 HTML 标签和文件系统非法字符，确保生成的文件名既保持可读性又符合系统规范。文件命名采用"发布日期_公告标题.pdf"的标准格式，便于后续的检索和管理，为数据的有序组织奠定了良好基础。

1.3.5 反爬虫与请求优化

为规避目标网站的反爬虫机制，本项目实施了多项请求优化策略。建立了丰富的 User-Agent 池，包含 Chrome、Firefox、Safari 等主流浏览器的多个版本标识，每次请求随机选择不同的 User-Agent，有效模拟了真实用户的访问行为。请求间隔采用随机化设计，在 3-10 秒范围内随机生成延迟时间，避免了固定频率请求被识别为爬虫程序。此外，还为 PDF 下载请求构建了真实的 Referer 头信息，进一步增强了请求的合法性和隐蔽性。

1.3.6 异常处理与日志监控

系统建立了完善的异常处理体系，对网络请求异常、JSON 解析异常、文件操作异常等各类错误进行了精细化捕获和处理。通过 try-except 块嵌套使用，确保了即使在某个处理环节出现异常，也不会影响整体采集流程的继续进行。日志系统采用双输出模式，既在控制台实时显示运行状态，又同步记录到文件中长期保存。日志内容涵盖从开始爬取到结束的完整生命周期，包括每页的处理状态、每个文件的下载结果以及所有异常情况的详细描述，为系统监控和故障排查提供了充分依据。

1.3.7 流程控制与性能优化

主循环流程采用了多条件控制机制，同时监控已下载文件数量和当前页码，确保在达到目标数量或最大页码限制时正常退出。参数组合索引和页码管理实现了智能联动，当某种参数组合无法获取数据时自动切换，当获取到有效数据时则保持当前组合继续翻页。这种动态调整策略最大限度地提高了数据采集的成功率和效率。通过设置合理的超时时间和连接参数，在保证请求成功率的同时，避免了因单个请求阻塞而影响整体采集进度。

1.4 总结

本项目的数据采集部分成功构建了一个高效、稳定的基金发行公告采集系统。通过综合运用 requests 会话管理、智能参数轮询、精准内容过滤和多重反爬虫策略，系统能够从巨潮资讯网可靠地获取目标 PDF 文档。采用流式下载与规范化的文件管理机制，既保障了数据完整性，又提升了存储效率。完善的异常处理和日志监控体系确保了系统长期运行的稳定性，为后续的结构化信息提取提供了高质量的数据基础。整个采集方案设计合理，具有较强的实用性和可扩展性，能够适应大规模数据采集的需求。

2.结构化提取技术详解

2.1 文本解析与预处理

2.1.1 路径与 OCR 设置

公告 PDF 既可能是“文本型”，也可能是“扫描型”。文本型优先用 PDF 文本解析，解析不到再走 OCR 兜底。脚本通过统一的路径与开关配置管理两条通路。

如何设置：

在 main.py 顶部修改：PDF_DIR、OUT_DIR、TESSERACT_EXE、POPPLER_BIN、ENABLE_OCR_FALLBACK。Windows 推荐写绝对路径；macOS/Linux 同理（注意 poppler 二进制路径）。

注意事项：

不想启用 OCR：将 ENABLE_OCR_FALLBACK=False；

启用 OCR 前请确认本机已安装 Tesseract 与 poppler，并把两个路径正确指向。

2.1.2 正文文本抽取

首选 pdfplumber 逐页抽取文本；失败或空文本时，回退到 PyPDF2。

若文本仍为空或中文字符极少（<20），再触发 OCR（pytesseract+pdf2image）。

脚本逻辑

extract_text(pdf_path)：先用 pdfplumber，再用 PyPDF2；返回拼接的全文字符串。

ocr_pdf_to_text(pdf_path)：以 300dpi 渲染页面→pytesseract.image_to_string(..., lang="chi_sim+eng")，并返回合并的 OCR 文本。

主循环中：若 full_text 为空或中文极少，且 ENABLE_OCR_FALLBACK=True，打印“启用 OCR 兜底”并调用 OCR。

常见问题

报错找不到 tesseract 或 poppler：请检查 TESSERACT_EXE/POPPLER_BIN 路径是否正确。

OCR 结果质量受扫描清晰度影响，可尝试：更高 dpi、裁切页眉页脚后再 OCR（可在脚本外预处理）。

2.1.3 文本标准化与正则友好化

PDF 会把中文拆字、插入全角空格/不可见空白，影响后续正则与标签定位。脚本提供统一的清洗与“无空白”工具函数。

关键函数：

clean_basic：去全角/不可见空白、压缩空格、trim。

collapse_cjk_spacing：合并被空白拆开的中文、规范冒号、压缩空白。

_nospace：移除所有空白字符，专门处理“基 金 管 理 人”这种断字场景。

2.2 表格优先的要素抽取

2.2.1 标签映射与键值对抽取

公告常以“键值表”的形式列出关键要素（如“基金管理人：××公司”）。不同公告对同一字段的称谓不统一（如“基金全称/基金简称/产品名称”）。脚本先将多种标签归一映射到统一字段名，再从“标签→右侧值”的相邻单元格中抽取。

标签映射表

TARGET_LABELS 将“基金全称/产品名称/基金类别/运作类型/托管人”等同义标签映射为标准字段（如“基金名称/基金类型/运作方式/基金托管人名称”）。如需扩展，新增键值即可。

抽取函数

extract_tables_keyvals(pdf_path)：逐页读取表格；

A) 识别“标签→右侧值”键值对；

B) 识别“费用类型丨费率/收费方式”表（见 2.2.2）；

返回：(kv 字典,表格纯文本拼接)，供后续兜底与正则搜索使用。

公司名跨行拼接:

管理人/托管人字段若出现“跨行/跨格”，脚本会“本行右邻→同行后续非标签格→下 1–2 行（若下一行第一格是新标签则停止）”，将片段粘在一起，并从合并文本里取**第一个“……公司”**作为值。

结果写入前调用 clean_org_name 做规整与校验（见 2.4）。

输出:中间结果存放于内存中的 kv 字典；后续 extract_fields(...)会结合 kv 与正文再做兜底与清洗。

2.2.2 费用表解析与标准化

认购/申购/赎回费率通常独立成表。表述可能为“≤/<=/不超过/不高于/0/不收取/暂无”。脚本统一映射为标准百分数或空值，确保下游易用。

关键实现:

_parse_fee_table(rows)：识别“费用类型丨费率/收费方式”表头，遍历行，将“认购/申购/赎回”三类费率填入结果。

_extract_rate_from_text(s)：0/0%→"0%"；

“不涉及/不收取/无/—/-/暂无”且无数字 → 设为空；

带“≤/<=/不超过/不高于 0.50（%可省略）”→ 统一成 "{前缀}{数值}%"。

扩展（分段认购费）:

若表格或正文出现“50 万≤S<100 万 0.5%/S≥100 万每笔 500 元”等分段描述，可在当前导出 CSV 的基础上，另行保存分段费率 JSON。脚本中已提供示例解析函数雏形供参考（parse_subscription_tiers）。

2.3 正文兜底（文本检索与规则融合）

表格缺失或不完整时，需要在正文段落中“搜索—定位—抽取—归一”。脚本提供通用的“正则+窗口”工具集，并对基金名称/代码、日期、类型/运作方式、三费率、募集规模分别做兜底。

2.3.1 名称与代码

步骤与要点

1）先从表格 kv 中取“基金简称/基金名称/产品名称”；失败再从正文与表格拼接文本中检索；最后从文件名兜底。

sanitize_fund_name：去“场内简称：…”尾巴、压掉“基金基金”、去尾部噪声。

postprocess_name_and_code：从名称中剥离“基金代码/交易代码：六位数”，若尾部直接跟 6 位数字也能识别为代码。

name_from_filename：从文件名清洗出可能的基金名（去日期/无关词/代码等），用于兜底。

文本检索工具

search_first(patterns,text)：按多组模式找首个命中。

search_following_value(lines_text,flat_text,labels)：遇到“基金名称/产品名称”等标签后，向下合并 1–5 行，直到凑成以“公司”结尾或遇到下一个标签；再用“扁平文本”兜底一次。

2.3.2 发行时间（日期归一）

做法:文本预处理后，通过 pick_start_date(s)/normalize_date(d)统一日期分隔符与格式，最终输出 YYYY-MM-DD。

2.3.3 基金类型与运作方式

先看表格映射；没有再用正文枚举正则：

strip_fund_type：提取“股票型/债券型/混合型/指数型/QDII/FOF/ETF”等，去掉“交易币种”等后缀噪声。

strip_operation_mode：提取“开放式/封闭式/交易型开放式（ETF）等”。

2.3.4 募集规模与三费率（正文兜底）

三费率：_clean_rate 与_extract_rate_from_text 组合处理“不收取/不涉及/0/%/≤/<=/不超过/不高于”等多种写法，统一成标准值或空。

募集规模：利用“规模/发行总额/上限”等关键词附近窗口检索并归一化为可读数值（如保留原文或解析出数额+单位）。

2.4 机构名称识别与角色消歧（管理人/托管人）

2.4.1 极端断字与跨行

问题场景:

“基金管理人：中国××基金管理有限公司的托管银行……”

标签在上一行，值在下一行；公司名内部又换行或插空格。

解决思路

跨行拼接：grab_company_multiline_after_label 把“标签行后 1–3 行”粘起来，去掉所有空白后，从中取第一个“……公司”。

去空白匹配：find_company_no_whitespace 将全文去空白后，用“无空白标签+……公司”一次性提取，专治“每字一空格”的排版。

2.4.2 银行词典与附近窗口

预置常见托管银行 BANK_SET，并生成“无空白正则”_bank_nospace_regex()，以适配“中国工商银行股份有限公司”这类文本。

efund_fastpath：若检测到“易方达基金管理”，直接认定管理人，同时在“基金托管人”标签附近±300 字优先匹配银行名，附近没有再全局匹配。

2.4.3 清洗与校验、角色纠偏

清洗：

clean_org_name：去标签词、合并断行/间隔、只保留第一个“……公司”、去“公司公司”、过滤明显脏值；要求至少含“银行/证券/基金/…/股份有限公司/有限责任公司”。

角色判别：

looks_like_bank：如果名称明显更像银行/券商托管侧，则辅助判别角色；结合“管理人/托管人”上下文标签进行自动纠偏。

find_org_after：在标签后 200 字内兜底抓取“……公司”。

2.5 字段后处理与一致性校验

从表格与正文拼凑出的字段，需统一格式与逻辑校验，保证 CSV 输出一致、可用。

规则清单

名称/代码：sanitize_fund_name → postprocess_name_and_code，名称去噪，代码 6 位提取、去重；若缺失，用 name_from_filename 兜底。

日期：pick_start_date/normalize_date 归一到 YYYY-MM-DD。

机构名：clean_org_name 校验关键信息词或“有限公司/股份有限公司”后缀，否则

置空。

三费率：统一百分比；0 → "0%"；“不收取/不涉及/—/-/暂无”→ 若无数字则置空，有数字则以数字为准。

合成与取舍策略：

优先使用表格结果；若表格缺失/为空，再用正文/表格文本拼接兜底；多个候选值冲突时选更长、更规范的那个（如公司名）。该策略体现在表格解析与字段合成的实现细节中。

2.6 结果导出与文件说明

运行方式：直接运行 python main.py，脚本会：

1）递归扫描 PDF_DIR；

2）抽取正文（必要时 OCR）与表格；

3）融合字段并清洗归一；

4）写出 CSV。

输出文件

OUT_DIR/基金公告_结构化提取.csv，字段顺序：

文件名,基金名称,基金代码,发行时间,基金管理人名称,基金托管人名称,基金类型,运作方式,募集规模,认购费率,申购费率,赎回费率（脚本中定义）。

字符集：utf-8-sig（带 BOM），Excel 双击可直接打开不乱码。

缺失值策略：无法从公告中确定的字段留空或写入“空”（与脚本策略一致），符合赛题“未涉及则为空”的要求。

2.7 使用建议与排错清单（可选增强）

OCR 太慢：仅在确有扫描件时打开 ENABLE_OCR_FALLBACK；或先人工筛分“疑似扫描件”；或分批跑。

机构名抽错到对方角色：检查正文是否存在“交叉贴近”场景；可先启用 efund_fastpath 的附近窗口匹配或扩大窗口范围。

公司名被拆字/断行：确保使用 find_company_no_whitespace 或 grab_company_multiline_after_label 这两类“去空白/跨行拼接”的兜底方法。

费率不统一：优先表格；正文出现“≤/不超过/0/不涉及”时按 _extract_rate_from_text 规则规范化。

日期格式混乱：统一走 pick_start_date/normalize_date，保证最终 YYYY-MM-DD。

与赛题字段一致性核对：本脚本输出的 11 个字段对齐赛题抽取清单，如公告未提及以空值表示，满足规范。

附：关键函数对照表（便于读代码与二次开发）

文本获取：extract_text（pdfplumber → PyPDF2），ocr_pdf_to_text（pytesseract+pdf2image）

文本清洗：clean_basic，collapse_cjk_spacing，_nospace

表格解析：extract_tables_keyvals，_parse_fee_table，TARGET_LABELS

正文搜索：search_first，search_following_value，normalize_date，pick_start_date

名称/代码：sanitize_fund_name，postprocess_name_and_code，name_from_filename

机构名与角色：clean_org_name，grab_company_multiline_after_label，find_company_no_whitespace，looks_like_bank，find_org_after，_bank_nospace_regex + BANK_SET

三费率：_extract_rate_from_text / _clean_rate（标准化“不收取/≤/0”等）

批处理与导出：walk_pdfs（递归扫描），main()（管线 orchestration，写出 CSV）